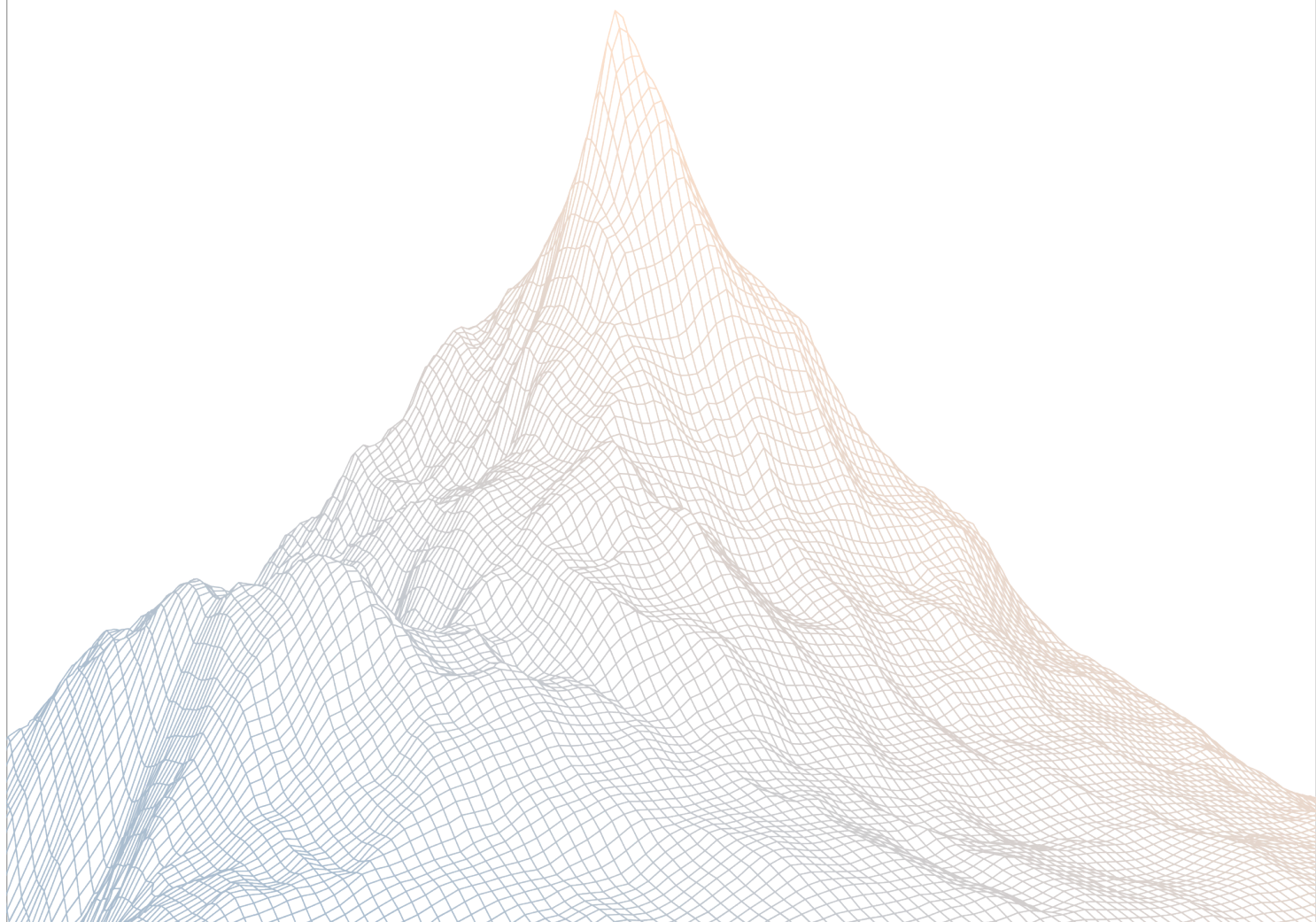


Solomon

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

August 25th to September 10th, 2026

AUDITED BY:

Dravee

Mario Ponder

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Solomon	4
2.2	Scope	4
2.3	Audit Timeline	6
2.4	Issues Found	6
3	Findings Summary	6
4	Findings	8
4.1	Medium Risk	9
4.2	Low Risk	18
4.3	Informational	38

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Solomon

Solomon builds infrastructure for productive dollars and programmable asset economics.

USDv is Solomon's liquid, reserve-backed onchain dollar. Eligible holders can earn while continuing to hold and use the same asset.

Businesses can integrate USDv and configure how its shared economics are retained, routed, split, or used to fund programs. Asset issuers can use the Solomon Platform to program the economics of their own assets.

2.2 Scope

The engagement involved a review of the following targets:

Target	chancery-audits
Repository	https://github.com/SolomonLabs/chancery-audits
Commit Hash	9eccca8dd10d307d634587d11b873f63a21fce60
Files	account_security.rs entrypoint.rs lib.rs processor.rs state_loader.rs modules/mod.rs modules/control/auth.rs modules/control/mod.rs modules/control/state/asset_pause_state.rs modules/control/state/mod.rs modules/control/state/module_activation_state.rs modules/control/state/pause_state.rs modules/core/instructions/mod.rs modules/core/mod.rs modules/core/state/asset_config.rs modules/core/state/chancery_config.rs modules/core/state/mod.rs modules/events_cpi/**/*rs modules/evidence/emit/limit.rs modules/evidence/emit/settlement.rs modules/evidence/event.rs modules/evidence/state/evidence_policy.rs modules/fees/mod.rs modules/fees/state/*rs modules/limits/dimension.rs modules/limits/mod.rs modules/limits/state/*rs modules/migration/instructions/*rs modules/migration/mod.rs modules/migration/state/*rs modules/pathway/mod.rs modules/pathway/state/*rs modules/permissions/auth.rs modules/permissions/mod.rs modules/permissions/state/*rs modules/reserve/instructions/mod.rs modules/reserve/instructions/withdraw_reserve.rs modules/reserve/mod.rs modules/reserve/state/*rs modules/settlement/instructions/*rs modules/settlement/mod.rs modules/settlement/state/*rs

2.3 Audit Timeline

August 25, 2026	Audit start
September 10, 2026	Audit end
September 10, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	3
Low Risk	9
Informational	13
Total Issues	25

3

Findings Summary

ID	Description	Status
M-1	Shared slot marker permits pre-aged module reactivation approvals	Resolved
M-2	Stale Observation Lets an Activated TransferHook Fire on a Collateral the Protocol Believed Safe	Acknowledged
M-3	Generationless configuration payloads allow pre-aged approvals to bypass fresh containment	Resolved
L-1	Fresh regrants of revoked non-pathway permissions remain unusably paused	Resolved
L-2	Pre-aged reserve-destination re-enablement approvals can bypass fresh containment	Resolved
L-3	forbidden_issued_token_extension_mask Enforced in Code, Marked Reserved in Docs	Resolved
L-4	Early Return in set_asset_mode Bypasses Access Control for No-Op Inputs	Resolved
L-5	Ambiguous Notional Denomination in Multi-Directional Settlement Policies	Acknowledged
L-6	Pause state does not preserve reliable emergency-containment provenance	Resolved
L-7	Settlement evidence omits governing dimension-policy identities and revisions	Resolved
L-8	Same-principal trilateral settlements cannot execute under counterparty daily caps	Resolved
L-9	Clearing an expired pause can re-arm stale bits and attribution	Resolved
I-1	Expired config changes can emit cancellation evidence	Resolved
I-2	Reserve-destination governance payload is an undocumented byte-order exception	Resolved

ID	Description	Status
I-3	Freeze policy documents advertise nonexistent flag-conditioned thaw restrictions	Resolved
I-4	The normative intent-ID formula omits the action byte enforced on-chain	Resolved
I-5	Corridor registration does not require non-zero caps, unlike withdrawal destinations	Resolved
I-6	Settlement intent creation omits current global, asset-pause, and asset-eligibility gates	Resolved
I-7	Local transfer-dependent routes can be activated for non-transferable collateral	Acknowledged
I-8	Rebate Rounding Mode Documented Inconsistently Across Spec Sections	Resolved
I-9	One raw-input maximum is reused across collateral and issued-token denominations	Acknowledged
I-10	External executor delegation violates compromised-executor containment guarantees	Resolved
I-11	Per-asset rate checks do not prevent cross-asset reserve arbitrage	Acknowledged
I-12	Range Check Allows Minimum Amount That Always Reverts	Resolved
I-13	The fee-policy validity predicate accepts the reserved zero ID	Resolved

4

Findings

4.1 Medium Risk

A total of 3 medium risk findings were identified.

[M-1] Shared slot marker permits pre-aged module reactivation approvals

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/control/state/module_activation_state.rs#L30-L50](#)
- [../src/modules/control/instructions/propose_config_change.rs#L102-L184](#)
- [../src/modules/control/instructions/set_module_status.rs#L90-L129](#)
- [../src/modules/control/instructions/set_module_status_with_pending_change.rs#L70-L85](#)
- [../src/modules/control/instructions/set_module_status_with_pending_change.rs#L160-L215](#)
- [spec/O2_Security_Posture_and_Authority_Control.md#L689-L705](#)

Description:

Module-status pending changes use `ModuleActivationState.last_updated_at_slot` as a generation marker. The marker is predictable, clock-derived, and shared by every module in the switchboard.

The generic proposal instruction accepts caller-supplied old and new hashes without verifying that they represent the module's current status and current marker. Governance can therefore create and mature a reactivation approval that commits to a selected future slot.

```
pub(crate) fn module_status_change_payload(  
    module_id: u8,  
    old_status: u8,  
    new_status: u8,  
    last_updated_at_slot: u64,  
) → [u8; 11] {  
    let mut payload = [0u8; 11];
```

```

payload[0] = module_id;
payload[1] = old_status;
payload[2] = new_status;
payload[3..].copy_from_slice(&last_updated_at_slot.to_le_bytes());
payload
}

```

Every real direct module transition replaces the shared marker with the transaction's current slot:

```

if old_status ≠ args.new_status {
    s_mut.module_statuses[args.module_id as usize] = args.new_status;
    s_mut.last_updated_by = *auth_account_info.key;
    s_mut.last_updated_at_slot = clock.slot;
}

```

A reachable sequence is:

1. Governance creates and accepts an approval to change module X from EMERGENCY_DISABLED to ACTIVE, committing to future slot S.
2. A later incident places X in EMERGENCY_DISABLED.
3. At slot S, current governance performs a real immediate restrictive transition on another module Y.
4. The transition updates the shared last_updated_at_slot to S while X remains EMERGENCY_DISABLED.
5. In the same transaction, governance consumes the pre-aged approval for X. The live status and shared marker now match its old hash, so X returns to ACTIVE.

The approval completed its timelock before the incident. Its consumption therefore bypasses a fresh post-containment review delay, contrary to the documented guarantee that intervening module transitions invalidate older approvals.

The transaction must land in the committed slot, which reduces reliability. Governance can improve reliability by preparing approvals for multiple future slots, while failed attempts roll back without consuming the pending record. The approval must also remain unexpired, and current governance must sign consumption.

Reactivation does not itself transfer value, and other live pauses, permissions, and policies continue to apply. However, when module status is the operative containment layer, premature reactivation can reopen settlement, reserve, permission, or cross-chain execution while the incident remains unresolved.

This differs from the *Generationless configuration payloads allow pre-aged approvals to bypass fresh containment* finding, which treated the module-status slot marker as an

effective defense. Here, that marker is itself pre-ageable and shared across modules; its remediation did not modify module activation or proposal-time validation.

Recommendations:

It is recommended to replace the shared slot marker with a checked monotonic generation and validate the module's live state when its proposal is created.

Prefer a separate generation for each module. A module-specific proposal path should load `ModuleActivationState`, derive the old hash from the module's current status and generation, and derive the new hash using the next generation. Every real direct or pending transition must increment the affected module's generation, while exact no-ops must preserve it.

```
let old_generation = state.module_generations[module_id as usize];
let new_generation = old_generation
    .checked_add(1)
    .ok_or(ChanceryError::ArithmeticOverflow)?;

let old_payload = module_status_change_payload(
    module_id,
    current_status,
    current_status,
    old_generation,
);
let new_payload = module_status_change_payload(
    module_id,
    current_status,
    proposed_status,
    new_generation,
);
```

A single switchboard generation may be used if storage constraints prevent per-module counters, but unrelated transitions would then invalidate outstanding proposals. Regardless of layout, a monotonic counter must be combined with proposal-time live-state validation; otherwise callers can precommit to a predictable future generation.

Solomon: Resolved with [@e6cbcc3a50...](#) by validating module-status proposals against the live switchboard target, current status, and current slot marker before creating the pending change.

Zenith: Verified.

[M-2] Stale Observation Lets an Activated TransferHook Fire on a Collateral the Protocol Believed Safe

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

Target:

- constants/extensions.rs (collateral_default_forbidden, DORMANT_TRANSFER_HOOK)
- asset_config.rs#319-333 (assert_observed_mask_allowed)
- asset_config.rs#351-357 (assert_extensions_fresh)
- settlement/instructions/mod.rs#363-404 (cpi_transfer_asset_in)
- settlement/instructions/mod.rs#507-545 (cpi_transfer_asset_out_with_decimals)

Description:

The guard that exists to reject hostile collateral can be bypassed by a permitted dormant-hook + stale-observation configuration.

Settlement gates on a cached `observed_extension_mask`. The default forbidden mask permanently rejects an **active** TransferHook (bit 0) but explicitly permits a **dormant** one (DORMANT_TRANSFER_HOOK, bit 21 — hook program id None). A Token-2022 mint authority can set that hook program id at any time — a standard mint operation — turning the extension active. The freshness gate (`assert_extensions_fresh`) is disabled when `max_extension_observation_age_slots = 0`, so a stale observation still reports the dormant bit and `assert_extensions_approved()` passes while the mint now carries an active hook that executes on every TransferChecked CPI.

Attack surface — available. The trigger is legitimate and non-malicious-admin: register a dormant-hook collateral (allowed), set `max_age = 0` (supported config), let the hook authority activate the hook (normal mint operation), and skip refresh. No forged signatures, no admin compromise.

Attack surface — closed (fragile). Two legs were studied and do not yield value under current Solana runtime constraints:

- *Reentrancy*: blocked by CPI depth limit (depth 4), not by state locking. During the transfer CPI, usage windows are unborrowed and unrecorded, `chancery_config` is held as a shared immutable borrow (non-exclusive), and no reentrancy guard exists. A reentrant settlement passes validation and limit checks against stale window state — only its own token CPI at depth 5 stops it. If the depth limit is ever raised, or if a future refactor moves evidence emission before the token CPIs, the reentrancy defense

disappears and the reentrancy path reaches value extraction.

- *Forwarded-signer delegate drain*: Token-2022 does not forward the initial sender's signature to the transfer hook. The hook cannot use the reserve authority's PDA signer to issue further transfers.

Attack surface — open. The live legs are availability and user-loss, not fund-drain, ranked by believability:

- *DoS (availability)*: a hook that reverts (or is upgraded to revert) bricks every `TransferChecked` through that collateral, freezing mint and redeem and locking the reserve. Realized with only a malicious/compromised hook program — no collateral authority needed. It bricks the token for everyone, not just Chancery, and is reversible by removing the hook.
- *Fee-on-transfer (user loss)*: a hook that deducts a fee on each transfer makes users receive less on mint and redeem. On redeem the principal's slippage floor bounds the loss (reverts if the fee is too high); on mint the minter is credited only for the net reserve delta, so the token stays collateralized. Protocol accounting is not broken; the loss is borne by users.
- *Delta inflation (conditional, least likely)*: `cpi_transfer_asset_in` mints issued tokens from the reserve's measured balance delta; a hook that credits the reserve from a source other than the minter's own funds (e.g., a hook holding the collateral mint authority) inflates the delta and under-collateralizes the issued token. This is the only leg that threatens solvency, but it is the least believable — it requires the hook to control external collateral, which is not routine for a transfer hook.

Add `DORMANT_TRANSFER_HOOK` to `collateral_default_forbidden`. One constant change. The entire attack surface — stale observation, dormant-to-active transition, all downstream impacts — is eliminated at the root. No collateral with any form of transfer hook is ever accepted.

The permissionless refresh and reentrancy guard are defense-in-depth. Valuable but not required to close this finding.

Recommendations:

Add `DORMANT_TRANSFER_HOOK` to `collateral_default_forbidden::MINT_FORBIDDEN_L0`. This prevents any hook-bearing collateral from being registered, eliminating the dormant-to-active bypass entirely. One constant change.

As defense-in-depth: make `refresh_asset_extension_observation` permissionless and require non-zero `max_extension_observation_age_slots` for collateral. This hardens against future unknown extension transitions beyond transfer hooks.

Solomon: Discussed this internally. Would keep it as it is, we might run cron job to monitor this.

Zenith: Acknowledged.

[M-3] Generationless configuration payloads allow pre-aged approvals to bypass fresh containment

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/core/state/asset_config.rs#L127-L156](#)
- [../src/modules/pathway/state/pathway_policy.rs#L151-L173](#)
- [../src/modules/limits/state/limit_policy.rs#L94-L110](#)
- [../src/modules/evidence/state/evidence_policy.rs#L109-L123](#)
- [../src/modules/control/pending_change.rs#L49-L157](#)
- [../src/modules/core/instructions/set_asset_mode.rs#L61-L111](#)
- [../src/modules/core/instructions/set_asset_mode_with_pending_change.rs#L99-L153](#)
- [tests/e2e/O2_permissions.test.ts#L940-L980](#)

Description:

The canonical pending-change payloads for AssetConfig, PathwayPolicy, LimitPolicy, and EvidencePolicy encode their current semantic fields but do not include a monotonic state generation.

For example, the in-scope AssetConfig payload includes the asset mode and other governed fields, but no value that distinguishes two separate occurrences of the same semantic state:

```
pub(crate) fn config_change_payload(&self) → Vec<u8> {
    let mut payload = Vec::with_capacity(210);
    payload.push(self.decimals);
    payload.push(self.mode);
    payload.extend_from_slice(&self.asset_flags.to_le_bytes());
    payload.extend_from_slice(self.asset_mint.as_ref());
    payload.extend_from_slice(self.asset_token_program.as_ref());
    payload.extend_from_slice(&self.primary_reserve_compartment_id);
    payload.extend_from_slice(&self.approved_extension_mask[0].to_le_bytes());
    payload.extend_from_slice(&self.approved_extension_mask[1].to_le_bytes());
    payload.extend_from_slice(&self.deposit_rate_e9.to_le_bytes());
    payload.extend_from_slice(&self.redeem_rate_e9.to_le_bytes());
    payload.extend_from_slice(&self.minimum_deposit_amount.to_le_bytes());
```

```

payload.extend_from_slice(&self.minimum_redeem_amount.to_le_bytes());
payload.extend_from_slice
    (&self.maximum_single_settlement_amount.to_le_bytes());
payload.extend_from_slice(&self.status_flags.to_le_bytes());
payload.extend_from_slice(&self.forbidden_extension_mask[0].to_le_bytes());
payload.extend_from_slice(&self.forbidden_extension_mask[1].to_le_bytes());
payload.extend_from_slice(&self.required_module_mask[0].to_le_bytes());
payload.extend_from_slice(&self.required_module_mask[1].to_le_bytes());
payload.extend_from_slice
    (&self.max_extension_observation_age_slots.to_le_bytes());
payload
}

```

Pending-change identifiers include a caller-selected proposer_nonce. Governance can therefore create multiple independent pending records for the same target, risk class, and semantic old and new values. Each record can complete its timelock and independently become accepted.

At consumption, the generic pending-change guard compares the stored hashes with hashes reconstructed from the target's current semantic value:

```

if &pending.old_value_hash != expected_old {
    return Err(ChanceryError::ConfigChangeHashMismatch.into());
}

if &pending.new_value_hash != expected_new {
    return Err(ChanceryError::ConfigChangeHashMismatch.into());
}

pending.assert_proposed_by_current_governance(current_governance_authority)?;

let clock = Clock::get()?;

pending.assert_not_expired(clock.unix_timestamp)?;

```

These checks establish semantic equality, current governance-key binding, and expiration. For the affected state types, they do not establish whether the target changed and later returned to the same semantic value.

A reachable asset-mode sequence is:

1. An asset is FROZEN.
2. Governance creates and accepts proposals A and B for FROZEN → ACTIVE, using different proposer nonces.
3. Governance consumes A and activates the asset.

4. A later incident causes emergency authority to execute the valid immediate ACTIVE → FROZEN transition.
5. `AssetConfig::config_change_payload` again produces the semantic old value stored in B because the payload contains `mode` but no `generation`.
6. Current governance can consume B immediately, reopening the asset without a new timelock measured from the containment event.

Equivalent state cycles can affect widening changes to limit, pathway, and evidence policy semantics when a direct restrictive update restores the exact earlier payload.

The issue does not grant governance authority. Current governance must still sign consumption. Each pending record is individually one-shot, but consuming A does not invalidate B. Cancellation, expiration, or replacement of the current governance key can prevent consumption of an unconsumed record.

Permission state demonstrates the intended protection. Its pending payload includes `permission_generation`, and existing tests confirm that an intervening semantic cycle invalidates a duplicate accepted proposal even when the roles later return to their previous value.

A stale approval can therefore bypass the fresh review period intended to follow new containment and accelerate renewed settlement or other policy-governed value movement during an incident.

Recommendations:

It is recommended to add a checked monotonic generation to each affected configuration state or semantic change family and include it in the canonical pending-change payload.

For each proposal, bind the old hash to the current generation and the new hash to the next generation:

```
let old_generation = current.config_generation;
let new_generation = old_generation
  .checked_add(1)
  .ok_or(ChanceryError::ArithmeticOverflow)?;

proposed.config_generation = new_generation;
```

Every real semantic mutation in that family must advance the generation, including direct restrictive transitions and accepted pending-change execution. Exact no-ops must leave it unchanged.

Where account layouts are fixed, allocate the generation from existing reserved headroom while preserving the declared account size and required alignment. Prefer explicit checked counters over timestamps or slots.

Use separate family-scoped generations when operational state is intentionally independent. For example, an extension-observation refresh or a pathway pause that a policy update preserves need not invalidate unrelated policy governance. Any direct restriction capable of restoring an earlier pending-change payload must advance the generation used by that payload.

Add tests in which two equivalent proposals are accepted, one executes, a restrictive transition restores the original semantic value, and consumption of the remaining proposal fails due to the changed generation. Limiting proposal lifetime and concurrent proposals for the same target and generation may be added as defense in depth but must not replace generation binding.

Solomon: Resolved with [@bae28efe7f...](#) by binding pending-change hashes to checked monotonic generations that advance on every real mutation of the affected configuration state.

Zenith: Verified.

4.2 Low Risk

A total of 9 low risk findings were identified.

[L-1] Fresh regrants of revoked non-pathway permissions remain unusably paused

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/permissions/instructions/revoke_permission.rs#L91-L139](#)
- [../permissions/instructions/upsert_permission_with_pending_change.rs#L160-L176](#)
- [../permissions/instructions/upsert_permission_with_pending_change.rs#L264-L282](#)
- [../src/modules/permissions/mutation.rs#L196-L227](#)
- [../src/modules/control/instructions/mod.rs#L25-L52](#)
- [../src/modules/migration/instructions/migrate_legacy_to_token2022.rs#L188-L196](#)

Description:

Revoking a permission leaves its canonical PDA initialized as a zero-role tombstone with `PERMISSION_FLAG_PAUSED` set. This marker distinguishes the revoked record from a never-created record and invalidates stale grant approvals.

A fresh regrant correctly loads the tombstone as existing state, validates a newly accepted pending change, restores the requested roles, and advances `permission_generation`. However, the shared write helper unconditionally preserves the record's existing flags:

```
let operational_permission_flags = record.permission_flags;

// ...
record.role_bits = value.role_bits;
record.permission_flags = operational_permission_flags;
```

The tombstone's pause marker therefore survives the regrant. Because `PermissionValue` and the `upsert` arguments contain no `permission_flags`, the governance ceremony cannot request that it be cleared. The resulting `PermissionUpserted` event accurately reports the

retained flag, but canonical role gates reject the restored permission with `PermissionPaused`.

The only instructions that can clear this flag require a `PATHWAY`-scoped record containing settlement roles and no unrelated capabilities. They cannot clear non-pathway records such as migration-, destination-, token-account-, or remote-domain-scoped permissions.

Consequently, an ordinary authorized revoke followed by a complete fresh regrant produces an unusable permission for the same subject and scope. Recovery requires another subject key where operationally possible or a program upgrade. Actor-authority fallbacks and key rotation prevent this from becoming a protocol-wide deadlock, but the defect can prolong an outage and prevents the intended same-subject recovery workflow.

Recommendations:

It is recommended to treat the paused bit on a revoked tombstone as a revocation marker rather than an independent operational pause. After validating and consuming a fresh accepted regrant, clear `PERMISSION_FLAG_PAUSED` when the prior record was a revoked tombstone and the new record restores nonzero roles.

```
let was_revoked_tombstone = current
  .as_ref()
  .is_some_and(|record| record.is_revoked_tombstone());

// Apply the accepted regrant.

if was_revoked_tombstone && record.role_bits != [0u64; 2] {
  record.permission_flags &= !PERMISSION_FLAG_PAUSED;
}
```

Continue preserving the flag when mutating an ordinary paused, capability-bearing record. Add regression coverage proving that a revoked non-pathway permission becomes usable after a fresh regrant while an ordinary operational pause survives unrelated permission mutations.

Solomon: Resolved with [@47915939d2...](#) by clearing the revocation-only pause marker when a fresh regrant restores roles while preserving operational pauses on live permissions.

Zenith: Verified.

[L-2] Pre-aged reserve-destination re-enablement approvals can bypass fresh containment

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/reserve/state/reserve_destination.rs#L66-L98](#)
- [../reserve/instructions/set_reserve_destination_status_with_pending.rs#L84-L129](#)
- [../src/modules/reserve/instructions/set_reserve_destination_status.rs#L70-L104](#)
- [../src/modules/control/instructions/propose_config_change.rs#L44-L69](#)
- [../src/modules/control/instructions/propose_config_change.rs#L127-L134](#)

Description:

ReserveDestination does not maintain a monotonic generation for status transitions. Pending re-enablement hashes consequently commit only to the current and requested status bytes:

```
let old_hash = compute_config_change_hash(
    change_kind::SET_RESERVE_DESTINATION_STATUS,
    reserve_destination_account_info.key,
    risk.as_u8(),
    &[current.status],
);
let new_hash = compute_config_change_hash(
    change_kind::SET_RESERVE_DESTINATION_STATUS,
    reserve_destination_account_info.key,
    risk.as_u8(),
    &[args.new_status],
);
```

The pending-change identifier incorporates a caller-selected proposer_nonce. Governance can therefore create and accept two independent records for the same DISABLED → ENABLED transition.

After one record enables the destination, emergency, operations, or governance can directly disable it as an immediate restrictive action. The destination then returns to the status represented by the second record. Because no generation distinguishes the two

occurrences of `DISABLED`, the remaining accepted record matches again and current governance can consume it without serving a fresh timelock after the containment event.

The record must remain unexpired and uncanceled, the governance authority must not have rotated, and current governance must sign its consumption. Re-enabling also does not itself withdraw collateral: applicable reserve pauses, withdrawal authorization, and mandatory limits remain enforced. Nevertheless, when destination disablement is the operative containment measure, the stale approval can prematurely restore a collateral-outflow route.

Recommendations:

It is recommended to add a checked monotonic `status_generation` to `ReserveDestination`, consuming reserved layout headroom without changing the account size. Include the current status and generation in the old-value payload and the requested status and next generation in the new-value payload.

Every non-idempotent status transition must advance the generation, including immediate restrictive disables and accepted pending-change execution. Exact no-ops should preserve it.

Additionally, proposal creation must load the target destination and validate its live status and generation. Generation binding alone prevents duplicate approvals for the current generation, but the generic proposal mechanism otherwise permits hashes to be prepared for a predictable future generation. A reserve-specific proposal path, or equivalent target-aware validation, should derive the hashes from live state and reject future-state precommitments.

Invalidate outstanding status-change records created under the previous hash format, and add regression coverage confirming that any intervening destination mutation invalidates an earlier accepted approval.

Solomon: Resolved with [@Oe80fe8d37...](#) by binding status approvals to a checked generation and validating proposal hashes against the destination's live status and generation.

Zenith: Verified.

[L-3] forbidden_issued_token_extension_mask Enforced in Code, Marked Reserved in Docs

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- pathway_policy.rs: forbidden_issued_token_extension_mask field comment
- settlement_instructions_mod.rs: assert_issued_token_extension_gates
- Spec §9.7: omits pathway.assert_issued_token_extensions_allowed() from settlement-validation list

Description:

The PathwayPolicy struct comment marks forbidden_issued_token_extension_mask as reserved and not enforced. It is in fact enforced: assert_issued_token_extensions_allowed() runs in every settlement handler via assert_issued_token_extension_gates(). The field is not required to be zero in assert_runtime_supported(), consistent with active enforcement. Spec §9.7 also omits the check.

Two potential consequences:

1. An operator sets the field expecting no effect and silently blocks all settlements on that pathway for issued-tokens carrying the flagged extension, with no indication the field was responsible.
2. An integration validating settlements against §9.7 skips this gate. Settlements it predicts will succeed will actually revert on-chain.

Recommendations:

Correct the PathwayPolicy struct comment. Add pathway.assert_issued_token_extensions_allowed() to the §9.7 settlement-validation list.

Solomon: Resolved with [@5e00e77a42...](#)

Zenith: Verified. The current state is "reserved future-proofing" as forbidden_issued_token_extension_mask is 0-initialized at registration and not exposed by update_pathway_policy, so the pathway-level issued-token extension check always passes.

[L-4] Early Return in `set_asset_mode` Bypasses Access Control for No-Op Inputs

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- `set_asset_mode.rs: handle` (early return before `assert_direct_config_change_allowed`)

Description:

`set_asset_mode::handle` returns `Ok(())` when `current_mode == new_mode`, before `validate_asset_mode_transition` and `assert_direct_config_change_allowed` are reached. Any caller regardless of authority receives a success response. No state is written and no evidence is emitted.

Two structural risks exist despite the current harmlessness:

1. The handler is documented as authority-gated for all inputs. If a developer adds code after the early return (for example, an evidence emission to log no-op calls for audit purposes) that code executes without authority validation. Any caller can then inject fabricated governance evidence into the event log. The bypass propagates silently because the developer assumes the authority check already passed.
2. Off-chain monitoring watching `set_asset_mode` invocations for unauthorized governance activity misses all calls hitting the early return. During an incident where unauthorized mode changes are the threat model, this creates a blind spot in detection.

Recommendations:

Move the no-op guard to after the authority and transition validation. The optimization is preserved while the access control invariant holds for all code paths.

Solomon: Resolved with [PR-104](#).

Zenith: Verified.

[L-5] Ambiguous Notional Denomination in Multi-Directional Settlement Policies

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- `settlement/instructions/mod.rs#L160-L193` — `assert_settlement_policy_for_terms`
- `settlement_policy.rs#L269-L279` — `assert_notional`
- `mint_delegated.rs#L401`, `mint_trilateral.rs#L415` — `pass intent.asset_amount`
- `redeem_delegated.rs#L391`, `redeem_trilateral.rs#L406` — `pass intent.issued_token_amount`

Description:

`SettlementPolicy` stores a single `min_notional / max_notional` pair with no denomination tag. Mint handlers pass `intent.asset_amount` (collateral units) to `assert_notional`; redeem handlers pass `intent.issued_token_amount` (issued token units). A policy allowing both directions via `allowed_settlement_modes` applies the same numeric bounds to incomparable quantities.

With `deposit_rate_e9 = 500_000_000` (0.5%) and `max_notional = 1000`: mints are capped at 1000 collateral (*1000exposure*), *redeems are capped at 1000 issued tokens (only 500 exposure at the current rate)*. An admin intending a symmetric \$1000 cap gets asymmetric enforcement with no on-chain signal.

Not exploitable — the contract executes as written. The risk is an admin misconfiguring a dual-direction policy that silently applies different economic limits per direction.

Risk Analysis

Impact is medium in the sense that misconfigured bounds could allow larger-than-intended settlements or block legitimate ones. Likelihood is low — requires admin to create a dual-direction policy without accounting for the denomination switch, and policies are typically direction-specific. The function parameter is named `asset_amount` even when receiving issued token amounts, compounding the confusion for future developers.

Step-by-Step Example

Protocol owner creates a settlement policy with `max_notional = 1_000_000_000` (1000 USDC) and `allowed_settlement_modes` permitting both mint and redeem. Asset rate is 0.5:1. Alice mints with 1000 USDC collateral — passes ($1000 \leq 1000$). Bob redeems 1000 issued tokens worth 500 USDC — also passes ($1000 \leq 1000$). The policy intended to cap exposure at 1000, *but Bob's redemption was only 500* of exposure. If the rate were 2:1 instead, Bob's 1000 issued tokens would represent \$2000 of exposure — exceeding the intended cap but passing the check.

Recommendations:

Reject dual-direction policies at registration: add a check in `assert_parameter_sanity` that disallows `allowed_settlement_modes` from containing both MINT and REDEEM. Force separate policies per direction. Eliminates the ambiguity with no settlement-time cost.

Solomon: All supported collateral uses a 1:1 rate, so mint and redeem limits represent the same value.

Zenith: Acknowledged in this case.

[L-6] Pause state does not preserve reliable emergency-containment provenance

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/control/state/pause_state.rs#L45-L72](#)
- [../src/modules/control/state/asset_pause_state.rs#L34-L57](#)
- [../src/modules/pathway/state/pathway_policy.rs#L107-L112](#)
- [../src/modules/permissions/state/permission_record.rs#L79-L103](#)
- [../src/modules/control/instructions/set_global_pause.rs#L84-L147](#)
- [../src/modules/control/instructions/set_pathway_pause.rs#L71-L108](#)
- [../src/modules/control/instructions/set_executor_pause.rs#L74-L110](#)
- [tests/e2e/29_basic_freeze_thaw.test.ts#L267-L300](#)
- [spec/O2_Security_Posture_and_Authority_Control.md#L265-L300](#)
- [docs/policy/O4_PAUSE_AND_INCIDENT_RESPONSE.md#L63-L79](#)

Description:

The pause-state models do not reliably distinguish emergency containment from ordinary operational pauses.

PauseState and AssetPauseState store aggregate pause bits and one `activated_by` value for the overall episode:

```
pub global_pause_bits: u64;

pub reason_code: u32;
pub _pad1: [u8; 4];

pub activated_by: Pubkey;
pub activated_at_slot: u64;
pub expires_at_slot: u64;
```

A single `activated_by` field is not sufficient to preserve per-bit emergency provenance. If emergency adds restrictions to an existing operations-originated episode, active-episode transition rules may preserve the original metadata. The resulting state cannot identify

which individual pause bits were imposed by emergency.

Pathway and party pauses retain even less provenance. PathwayPolicy stores the pause only inside status_flags, while executor and counterparty pauses use only PermissionRecord.permission_flags. Neither state records who imposed the restriction or whether it belongs to an emergency-containment episode.

The supporting pause handlers authorize clearing solely from the current signer class:

```
let is_governance = authority_account_info.key
    == &chancery_config.governance_authority;
let is_ops = authority_account_info.key
    == &chancery_config.operations_authority;
let is_emergency = authority_account_info.key
    == &chancery_config.emergency_authority;

if args.is_clear && !(is_governance || is_ops) {
    return Err(ChanceryError::InsufficientRole.into());
}

if !args.is_clear && !(is_governance || is_ops || is_emergency) {
    return Err(ChanceryError::InsufficientRole.into());
}
```

The clear path does not inspect activated_by or any persistent emergency-origin bitmask. Pathway, executor, and counterparty handlers apply the same authority split despite having no provenance field to inspect.

Consequently, if an operations key is suspected or compromised, emergency authority can pause a global, asset, pathway, executor, or counterparty surface, but the same operations key can clear that containment in a subsequent transaction. Existing tests demonstrate the sequence by having emergency set an indefinite global pause and operations clear it directly.

The behavior matches the incident-response policy that allows operations or governance to clear these local pauses. It conflicts with the stricter security posture under which emergency restriction is immediate while relaxation after emergency containment is slower or requires stronger authority. The state model cannot implement both policies because it cannot distinguish an ordinary operations-originated pause from emergency containment that must survive operations-key compromise.

Clearing a pause does not independently grant settlement or reserve authority, and all remaining permission, pathway, asset, and limit checks continue to apply. Emergency may also use applicable module-level containment. Nevertheless, immediate clearing defeats the ordinary pause surfaces during the operations-key compromise scenario they are intended to contain.

Recommendations:

It is recommended to first select one canonical pause-recovery policy and align the state model, handlers, tests, and operational documentation with it.

If emergency containment must survive operations-key compromise, add explicit per-bit emergency provenance and a checked pause generation to each pause scope. For example:

```
pub emergency_pause_bits: u64,  
pub pause_generation: u64,
```

Whenever emergency adds a pause bit, that bit must be added to `emergency_pause_bits`, including when it is added to an existing operations-originated episode. This must not depend on whether general episode metadata is replaced.

Direct operations clearing should reject any requested bit present in `emergency_pause_bits`. Clearing or shortening emergency-originated restrictions should require governance and the configured pending-change ceremony. The accepted change should bind the exact effective bits, emergency bits, expiry, and pause generation.

Ordinary operations-originated pauses may remain immediately clearable by operations. Exact no-ops should not advance the generation, while every real set, clear, expiry change, or provenance change should advance it with checked arithmetic.

Use reserved state headroom where possible while preserving account sizes and alignment. Pathway and party pauses should receive equivalent dedicated provenance fields or use separate canonical pause-state PDAs rather than overloading policy flags.

If automatic expiry of emergency containment is allowed, document it as an intentional relaxation mechanism and bind it to the emergency state. Otherwise, emergency-originated bits should not expire without the stronger clear procedure.

Add coverage for emergency additions to fresh and already-active operations episodes, partial per-bit clears, pathway and party pauses, stale pending clears, expiry behavior, and attempted clearing by a compromised operations authority.

Solomon: Resolved with [@1ca50934a6...](#) by clarifying that ordinary pauses are operations-clearable and directing authority-compromise response to durable module or asset containment.

Zenith: Verified.

[L-7] Settlement evidence omits governing dimension-policy identities and revisions

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/evidence/emit/settlement.rs#L183-L301](#)
- [../src/modules/pathway/state/pathway_policy.rs#L98-L143](#)
- [../src/modules/settlement/instructions/mint_delegated.rs#L597-L674](#)
- [../src/modules/settlement/instructions/mint_delegated.rs#L905-L917](#)
- [../src/modules/evidence/emit/pathway.rs#L10-L52](#)
- [../src/modules/evidence/emit/limit.rs#L15-L25](#)
- [../src/modules/evidence/emit/limit.rs#L143-L157](#)
- [../src/modules/evidence/emit/fee.rs#L10-L50](#)
- [../src/modules/evidence/emit/evidence_policy.rs#L13-L54](#)
- [../src/modules/evidence/emit/asset.rs#L10-L55](#)
- [spec/04_Settlement_Pathways_and_Permissions.md#L384-L386](#)
- [spec/07_Evidence_Reporting_Document_Alignment_and_Migration.md#L3-L13](#)

Description:

A pathway can reference a generic limit policy and up to four independent dimension policies. Settlement handlers load and enforce the applicable dimension policies, but SettlementMint and SettlementRedeem emit only the generic pathway `limit_policy_id`:

```
// PathwayPolicy contains the complete configured limit-policy set.
pub limit_policy_id: [u8; 32],
pub asset_mint_limit_policy_id: [u8; 32],
pub asset_redeem_limit_policy_id: [u8; 32],
pub counterparty_limit_policy_id: [u8; 32],
pub executor_limit_policy_id: [u8; 32],

// SettlementMint and SettlementRedeem contain only:
pub limit_policy_id: [u8; 32],
```

Mint settlements can enforce the asset-mint policy, redeem settlements can enforce the asset-redeem policy, and delegated or trilateral settlements can additionally enforce

counterparty and executor policies. Delegated pathways must enforce a counterparty-scoped policy with a nonzero daily maximum. Nevertheless, none of these dimension-policy IDs appears in the resulting settlement event.

The events do include the top-level settlement, generic limit, evidence, fee, and insurance policy IDs, together with the participants, token accounts, amounts, assessed fee, rebate, and measured economic outcome. The omission therefore affects the completeness of the governing policy set rather than the correctness of the settlement outcome.

Policy identity also does not establish the exact governing revision when policies are mutable. Pathway, limit, fee, evidence, and asset registration events omit material semantic fields, while their update events generally publish old and new hashes without the corresponding semantic preimages. Event payloads alone therefore cannot reconstruct historical caps, fee schedules, asset rates, pathway references, extension-policy masks, or evidence requirements after those accounts are modified.

A complete archive of successfully committed registration and update instruction data from initialization onward, or account snapshots at the relevant historical slots, can reconstruct additional information. Neither artifact is currently required by the event decoder specification. If an archive retains only decoded events, it cannot determine the complete policy set and exact policy revision that governed a historical settlement.

On-chain policy enforcement remains correct, and realized economic outcomes remain observable. The impact is limited to historical cap reconciliation, audit evidence, regulatory reporting, and incident analysis. This conflicts with the specification requiring emitted evidence to reconstruct which policy set governed an operation.

Recommendations:

It is recommended to append the following fields to `SettlementMint` and `SettlementRedeem`, using zero for dimensions that were not applicable or enforced:

```
pub asset_mint_limit_policy_id: [u8; 32],
pub asset_redeem_limit_policy_id: [u8; 32],
pub counterparty_limit_policy_id: [u8; 32],
pub executor_limit_policy_id: [u8; 32],
```

If event-only reconstruction remains a protocol requirement, each settlement should also bind the exact semantic revisions actually consumed. This may use immutable semantic revision IDs, individual canonical semantic hashes, or one canonical `governing_policy_set_hash`. This revision mechanism must not reuse the account layout version.

Registration events should publish the complete initial canonical semantic value, and update events should publish the complete resulting semantic value while retaining the existing governance hashes. The canonical preimages must exclude reserved bytes and unrelated volatile state.

Alternatively, if reconstruction is intended to depend on full chain history, the specification and decoder requirements should explicitly require archival of successfully committed outer instructions from initialization onward and any necessary historical state snapshots. Event schemas should remain append-only, and the archival requirements for existing schemas should be documented.

Solomon: Resolved with [@ce8068e490...](#) by appending all dimension-policy IDs to settlement evidence and defining archival requirements for reconstructing exact mutable-policy revisions.

Zenith: Verified.

[L-8] Same-principal trilateral settlements cannot execute under counterparty daily caps

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/settlement/instructions/create_settlement_intent.rs#L187-L200](#)
- [../src/modules/settlement/instructions/mint_trilateral.rs#L134-L175](#)
- [../src/modules/settlement/instructions/mint_trilateral.rs#L649-L690](#)
- [../src/modules/settlement/instructions/mint_trilateral.rs#L900-L910](#)
- [../src/modules/settlement/instructions/redeem_trilateral.rs#L134-L165](#)
- [../src/modules/settlement/instructions/redeem_trilateral.rs#L672-L712](#)
- [../src/modules/limits/dimension.rs#L45-L50](#)
- [tests/e2e/O6_settlement_trilateral.test.ts#L515-L524](#)
- [spec/O4_Settlement_Pathways_and_Permissions.md#L249-L258](#)
- [spec/I6_Security_Remediation_Invariants.md#L124-L142](#)

Description:

Trilateral intent creation permits one identity to occupy both principal roles:

```
match args.settlement_mode {
  settlement_mode::DELEGATED_NON_CUSTODIAL => {
    // Delegated-specific identity validation.
  }
  settlement_mode::TRILATERAL_ATOMIC => {}
  _ => return Err(ChanceryError::IntentInvalidParameters.into()),
}
```

Tests confirm that `principal_a = principal_b` is accepted and can execute when no counterparty daily window is required. This follows the explicit role-overlap model under which trilateral roles remain semantically distinct but may be occupied by the same identity.

When the pathway references a counterparty policy with `per_day_maximum` $\neq 0$, both trilateral handlers instead require separate daily-window account positions for principal A and principal B:

```
COUNTERPARTY_A_DAILY_USAGE_WINDOW,  
COUNTERPARTY_B_DAILY_USAGE_WINDOW,
```

Each expected window is derived as `sha256(COUNTERPARTY || principal_key)`. Equal principals therefore derive the same canonical window PDA.

Both window positions are included in the handlers' initial distinct-account matrix. Passing the canonical window at both positions fails with `AccountAliasNotAllowed` before the intent or policy is evaluated. Passing a different account at either position later fails canonical PDA verification, while omitting either required account fails with `MissingAccount`. No account arrangement can therefore execute the otherwise valid same-principal intent.

The issue is conditional on a nonzero daily counterparty cap. A counterparty policy containing only a per-transaction cap does not require these windows and does not trigger the conflict.

The documentation is internally inconsistent: one pathway specification describes the two principals as distinct identities, while the remediation invariants explicitly permit valid trilateral role overlap and require independent usage-window writes, not overlapping semantic roles, to remain distinct. Current validation and tests implement the overlap-permitted model, but the daily-window path does not.

No assets move because execution fails before state mutation or token CPI. The impact is limited to settlement liveness, failed transactions, and intent-rent or operational overhead until the intent is cancelled or becomes eligible for expiry cleanup.

Recommendations:

It is recommended to select one canonical trilateral identity model and align validation, account handling, tests, and documentation with it.

Under the currently implemented overlap-permitted model, handle an equal-principal settlement as one concrete counterparty:

- require one canonical counterparty daily window;
- require the second window position to use the default placeholder;
- perform the daily-cap check once; and
- accrue the settlement amount once.

For distinct principals, retain the existing requirement for two separately verified windows and accrue the full notional once to each principal's window.

The early distinct-account validation must account for this semantic overlap. It should continue rejecting aliases when the principals differ, while accepting the default second-window placeholder only after the fixed principal account keys indicate the same identity. Subsequent intent validation must still prove that those account keys match the committed intent.

Do not merely permit the same window account at both existing positions. The current recording logic writes through both prepared windows and would double-count the settlement amount.

Add mint and redeem coverage proving that:

- an equal-principal trilateral settlement with a nonzero daily cap succeeds;
- its shared counterparty window is checked and incremented exactly once;
- distinct principals still require and update two different windows; and
- an unexpected window alias for distinct principals remains rejected.

If the protocol instead adopts a strict distinct-principal model, reject `principal_a = principal_b` unconditionally at both intent creation and execution, and remove the conflicting overlap-permitted behavior from specifications, clients, and tests. The rejection should not depend on the current policy because policy bindings and caps can change while an intent remains pending.

Solomon: Resolved with [@2fcb0793f3...](#) by requiring distinct trilateral principal identities during intent creation and both execution paths while preserving executor/principal overlap.

Zenith: Verified.

[L-9] Clearing an expired pause can re-arm stale bits and attribution

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/control/pause_transition.rs#L19-L79](#)
- [../src/modules/control/pause_transition.rs#L229-L301](#)
- [../src/modules/control/state/pause_state.rs#L183-L215](#)
- [../src/modules/control/state/asset_pause_state.rs#L183-L194](#)
- [../src/modules/control/instructions/set_global_pause.rs#L109-L146](#)
- [../src/modules/control/instructions/set_global_pause.rs#L158-L189](#)
- [../src/modules/control/instructions/set_asset_pause.rs#L59-L108](#)
- [../src/modules/control/instructions/set_asset_pause.rs#L242-L274](#)
- [spec/16_Security_Remediation_Invariants.md#L236-L245](#)

Description:

Global and asset pause bits remain stored after automatic expiry. Runtime enforcement nevertheless treats the complete episode as inactive once the current slot reaches the stored expiry:

```
if self.expires_at_slot  $\neq$  0 && current_slot  $\geq$  self.expires_at_slot {  
    return Ok(());  
}
```

The shared set transition detects this inactive state and starts a fresh episode. The clear transition does not perform the equivalent activity check. It removes the requested bits from the raw stored bitfield and, when at least one bit survives, accepts a new future expiry:

```
let new_bits = current_bits & !requested_bits;  
  
if new_bits == 0 {  
    return Ok(PauseClearTransition {  
        new_bits: 0,  
        new_expires_at_slot: 0,  
        canonicalise: true,  
    });  
}
```

```
});  
}  
  
let new_expires_at_slot = if requested_expires_at_slot == 0 {  
  current_expires_at_slot  
} else if requested_expires_at_slot > current_slot {  
  requested_expires_at_slot  
} else {  
  return Err(ChanceryError::PauseStateInvalidParameters.into());  
};
```

After an episode expires, governance or operations can therefore submit a clear request that leaves at least one stale bit and supplies a nonzero expiry greater than the current slot. The surviving bits become effective again even though the instruction was classified as a clear.

A zero-bit clear preserves and reactivates every stale bit. Because the raw bitfield does not change, the event reports `changed_pause_bits = 0` despite moving from an effectively unpaused state to an effectively paused state.

The transition is not marked for canonicalization, so the handlers retain the expired episode's stored opener, activation slot, and reason. The emitted event combines:

- `is_clear = true`;
- the new future expiry;
- the reactivated bits as `effective_pause_bits`;
- the current request's `reason_code`;
- the expired episode's opener as `activated_by`.

This creates inconsistent episode attribution: state retains the previous reason while evidence reports the current clear reason, and the old opener remains attached to newly effective containment.

Only governance or operations can execute the clear path, and both can directly set pauses. This is not an authority escalation. The impact is unexpected reactivation of global or asset restrictions and misleading episode provenance during legitimate clear or expiry-adjustment operations.

Pathway and permission-record pauses are unaffected because they do not use the shared expiring episode model.

Recommendations:

It is recommended to determine effective activity before processing any clear. When the stored episode is inactive or expired, canonicalize it instead of applying requested bits or expiry changes to stale state:

```
let current_is_active = current_bits ≠ 0
  && (current_expires_at_slot = 0 || current_slot <
    current_expires_at_slot);

if !current_is_active {
  return Ok(PauseClearTransition {
    new_bits: 0,
    new_expires_at_slot: 0,
    canonicalise: true,
  });
}
```

The existing handler canonicalization branches should then clear the stored reason, opener, activation slot, bits, and expiry.

Preserve expiry adjustment for partial clears only while the current episode remains active. After expiry, an authority wishing to establish containment should use the set path so the action records a fresh reason, opener, activation slot, and non-clear event.

If reactivation through the clear instruction is intentionally retained, classify it as a fresh set transition and replace the expired episode's metadata. It should not emit `is_clear = true` or report zero changed bits while effective containment is introduced.

Add unit and integration coverage for:

- clear at and after the expiry boundary;
- zero-bit, partial-bit, and full-bit clear requests;
- future expiry supplied against an inactive episode;
- canonical clearing of bits, expiry, opener, activation slot, and reason;
- emitted effective bits, changed bits, action classification, and attribution;
- unchanged partial-clear and expiry-adjustment behavior for active episodes.

Solomon: Resolved with [@5922bdf964...](#) by canonicalizing inactive or expired global and asset pause episodes before any clear operation, preventing stale bits and attribution metadata from being re-armed.

Zenith: Verified.

4.3 Informational

A total of 13 informational findings were identified.

[I-1] Expired config changes can emit cancellation evidence

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/control/instructions/cancel_config_change.rs#L90-L109](#)
- [../src/modules/control/instructions/close_expired_config_change.rs#L82-L110](#)

Description:

`cancel_config_change` allows governance to close a pending change in `PROPOSED` or `ACCEPTED` status without checking whether its expiration time has passed:

```
match p.status {
  config_change_status::PROPOSED | config_change_status::ACCEPTED => {}
  _ => return Err(ChanceryError::NotInitialized.into()),
}
```

Meanwhile, `close_expired_config_change` allows anyone to close the same record after expiration:

```
if clock.unix_timestamp < p.expires_at_unix_timestamp {
  return Err(ChanceryError::ConfigChangeNotExpired.into());
}
```

Consequently, governance can cancel an already-expired change before a permissionless expiry sweep executes. The resulting terminal event depends on which transaction executes first: governance produces `ConfigChangeCancelled`, while the permissionless path produces `ConfigChangeExpired`.

This cannot execute an expired change, duplicate its refund, or produce both events. Solana account locking serializes the transactions, and closing the account causes

subsequent attempts to fail. The proposal event also preserves the expiration timestamp, so a full-history observer can reconstruct that cancellation occurred after expiry.

The impact is limited to inconsistent terminal classification for off-chain workflows that interpret cancellation as an explicit pre-expiry withdrawal and expiration as a natural lapse.

Recommendations:

It is recommended to make the lifecycle outcomes disjoint by rejecting cancellation after expiration using the existing state helper:

```
p.assert_not_expired(clock.unix_timestamp)?;
```

This preserves permissionless cleanup through `close_expired_config_change` while ensuring expired records consistently emit expiration evidence.

Solomon: Resolved with [@6488e4561e...](#) by rejecting cancellation once a pending change reaches its expiration timestamp, leaving expiry as the sole terminal path.

Zenith: Verified.

[I-2] Reserve-destination governance payload is an undocumented byte-order exception

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/reserve/instructions/register_reserve_destination_with_pending.rs#L185-L191](#)
- [../src/modules/core/state/asset_config.rs#L135-L145](#)
- [tests/e2e/12_reserve.test.ts#L127-L138](#)
- [scripts/mainnet/PrepareMainnetActivation.ts#L683-L685](#)

Description:

Comparable local pending-change semantic payloads for asset, limit, fee, evidence, permission, and module-status configuration encode multi-byte integer fields as little-endian. Reserve-destination registration is an exception: its custom 72-byte hash preimage encodes `destination_flags` as a big-endian u64.

```
// the complete mutable semantic value (owner, purpose, withdrawal policy)
// is committed into the pending-change payload.
let mut new_payload = [0u8; 72];
new_payload[0..32].copy_from_slice(args.destination_owner.as_ref());
new_payload[32..40].copy_from_slice(&args.destination_flags.to_be_bytes());
new_payload[40..72].copy_from_slice(&args.withdrawal_limit_policy_id);
```

This is a custom, per-change-kind hash preimage, so Borsh's little-endian convention does not apply and no protocol-wide rule requires the field to be little-endian. Existing tests explicitly treat the big-endian representation as canonical, and the mainnet activation tooling reproduces it correctly. There is therefore no present hash mismatch, authorization bypass, or incorrect state transition.

However, the exception is not specified in the protocol documentation, and its encoding is independently duplicated across implementations. Future ceremony tooling modeled on the comparable little-endian configuration payloads could calculate a different `new_value_hash`. If governance proposed and accepted that hash, consumption would fail closed because the handler recomputes the payload using big-endian bytes. The resulting impact is limited to an unusable pending change and a repeated governance ceremony.

Recommendations:

It is recommended to document the canonical payload as:

```
destination_owner[32]
|| destination_flags[u64 big-endian]
|| withdrawal_limit_policy_id[32]
```

Implement dedicated encoders in each supported language and validate them against shared fixed payload and hash vectors. Preserve the existing byte order unless the on-chain handler and every ceremony implementation are deliberately updated together with outstanding proposals accounted for.

Solomon: Resolved with [@251d932b44...](#) by documenting the canonical 72-byte reserve-destination payload and its explicit big-endian flags encoding.

Zenith: Verified.

[I-3] Freeze policy documents advertise nonexistent flag-conditioned thaw restrictions

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/control/instructions/thaw_issued_token_account.rs#L151-L200](#)
- [spec/O6_Reserve_Compartments_Provenance_and_Enforcement.md#L166-L172](#)
- [docs/policy/O3_FREEZE_AND_ENFORCEMENT.md#L25-L37](#)
- [docs/policy/O4_PAUSE_AND_INCIDENT_RESPONSE.md#L54-L67](#)

Description:

The freeze specification and operating policy state that governance, operations, or enforcement may thaw an account depending on configured control flags. The program defines no recognized freeze-flag vocabulary and does not use the stored flags when authorizing thaw.

`thaw_issued_token_account` authorizes governance, operations, enforcement, or a holder of a valid `CAN_THAW_TOKEN_ACCOUNT` permission. Afterward, it checks only that the freeze record has `FROZEN` status:

```
// Authority list: governance | ops | enforcement (NOT emergency).
let is_actor =
  authority_account_info.key == &chancery_config.governance_authority
  || authority_account_info.key == &chancery_config.operations_authority
  || authority_account_info.key == &chancery_config.enforcement_authority;

// ...
if !is_actor {
  let permission_record = assert_can_thaw_token_account(
    authority_account_info,
    &accounts[PERMISSION_RECORD],
    issued_token_account_info.key,
    &crate::id(),
  )?;
}
```

The handler does not inspect `frozen_by`, `freeze_flags`, or any origin-sensitive policy before executing the thaw CPI. Consequently, `freeze_flags` cannot restrict which otherwise-authorized thaw actor may release an account.

The current incident-response policy and test suite treat this unconditional actor list as intentional. Therefore, this does not establish an authorization bypass relative to the implemented model. Instead, the conflicting documentation may cause operators to believe that placing particular values in `freeze_flags` creates stronger recovery restrictions when those values currently function only as uninterpreted metadata.

The state layout also contains `thaw_flags`, but thaw closes the freeze record without writing that field. The caller-supplied value survives only in the terminal thaw event.

Recommendations:

It is recommended to align the freeze specification and operating policy with the implemented authority model by:

- removing statements that thaw authority depends on configured flags;
- documenting the unconditional governance, operations, enforcement, and scoped-permission authorization model;
- defining `freeze_flags` and `thaw_flags` as evidence-only metadata or reserved fields; and
- documenting that thaw closes the state record while the signed events preserve the freeze lifecycle.

If flag-conditioned recovery restrictions are intended instead, define recognized flag bits, restrict which authorities may set protected flags, reject unknown bits, and enforce the stored policy before the thaw CPI.

Solomon: Resolved with [@ea5de5ae52...](#) by documenting the implemented thaw authority model and defining freeze and thaw flags as evidence-only metadata.

Zenith: Verified.

[I-4] The normative intent-ID formula omits the action byte enforced on-chain

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/settlement/instructions/create_settlement_intent.rs#L107-L130](#)
- [../src/modules/settlement/instructions/create_settlement_intent.rs#L479-L498](#)
- [spec/13_PDA_Register.md#L122-L135](#)
- [tests/support/intentHash.ts#L90-L112](#)

Description:

The normative PDA register omits `settlement_action` from the canonical `intent_id` preimage. The on-chain implementation and repository TypeScript helper both include this byte immediately after `settlement_mode`.

```
hashv(&[
  domain_separator,
  &args.pathway_id,
  &[args.settlement_mode],
  &[args.settlement_action],
  args.principal_a.as_ref(),
  // ...
])
```

In contrast, the specification defines:

```
domain_separator || pathway_id || settlement_mode: u8 ||
principal_a || principal_b || ...
```

The handler requires both caller-supplied `intent_id` and `intent_hash` to equal the implementation's recomputed digest. It then derives the settlement-intent PDA from that value. A client implementing the normative formula therefore derives a different hash and PDA and is deterministically rejected with `IntentHashMismatch`.

Indexers and signing integrations following the documented formula may similarly derive an address inconsistent with the program and fail to locate or submit the intended intent.

The discrepancy does not allow malformed intent creation, incorrect signature acceptance, or asset loss; its impact is limited to integration failure and inconsistent intent identification.

Recommendations:

It is recommended to add `settlement_action: u8` immediately after `settlement_mode` in the normative formula in `spec/13_PDA_Register.md`.

Provide the canonical hash function through the supported client surface rather than only test infrastructure. Add a shared known-answer vector containing the complete preimage bytes, resulting digest, and derived PDA, and validate it from both Rust and TypeScript.

Avoid removing the action byte from the implementation solely to match the stale formula, as this would change the established intent namespace and invalidate any identifiers or signatures already prepared using the current convention.

Solomon: Resolved with [@942f23aaac...](#) by adding `settlement_action` to the normative intent-ID preimage so it matches the on-chain and TypeScript implementations.

Zenith: Verified.

[I-5] Corridor registration does not require non-zero caps, unlike withdrawal destinations

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- `modules/cross_chain/instructions/consume_inbound_message.rs`
- `modules/cross_chain/instructions/register_remote_domain_policy.rs`
- `modules/cross_chain/state/remote_domain_policy.rs`
- `modules/settlement/instructions/redeem_direct.rs`, `redeem_delegated.rs`, `redeem_trilateral.rs`
- `modules/reserve/instructions/register_reserve_destination_with_pending.rs` (comparison point)

Description:

Cross-chain corridors and reserve withdrawal destinations both affect the reserve's capacity to honor redemptions, but they have different guardrails at registration.

When you register a withdrawal destination, the program requires a bound limit policy with non-zero per-transaction and daily caps. You cannot create an uncapped withdrawal destination.

When you register a cross-chain corridor, the program accepts `per_message_maximum = 0` and `per_day_maximum = 0`, meaning uncapped. There is no floor.

To understand why this matters, here is how cross-chain minting works. A user locks or burns collateral on a remote chain. A quorum of attestors signs off on this. Then `consume_inbound_message` mints issued tokens locally. No collateral enters the local reserve during this process. The minted tokens are fungible with locally deposited ones, so holders can redeem them against the local reserve through the normal redeem paths (`redeem_direct`, `redeem_delegated`, `redeem_trilateral`).

The program does not track how much issued supply came from remote mints versus local deposits. The spec documents this: backing is an operational responsibility, not an on-chain guarantee, and the formal verification section states the program performs no issued-supply surplus check.

An uncapped corridor lets redeemable claims accumulate without bound. Redemption then draws the reserve down at whatever rate the redeem-side controls allow (pathway

caps, per-tx limits, pause bits), until it is exhausted. Redemption is role-gated on the pathway, so a run would be executed by permissioned counterparties (e.g. market makers absorbing stranded supply), which intermediates but does not prevent it. The most realistic trigger is not an attack but a corridor disruption: if a corridor is paused, retired, or broken, remote-origin holders have local redemption as their only exit, since their collateral is locked or burned on the far side. A concentrated redemption run by stranded holders is a plausible path from uncapped corridor to reserve exhaustion.

This is the honest-quorum case. It is distinct from the known issue "Outbound reclaim leaves a compromised quorum's mint blast radius uncapped," which covers a compromised quorum scenario. The spec anticipates uncapped corridors (its retirement rules handle "a newly finite per-message or absolute daily cap"), so the recommendation is a registration floor, not a redesign.

The cross-chain module is `DISABLED` at MVP. None of this is reachable today.

Recommendations:

Require non-zero `per_message_maximum` and `per_day_maximum` at corridor registration, matching the existing withdrawal-destination requirement. This does not change the backing model or add new invariants. It closes the gap so both surfaces that affect the reserve's capacity to honor redemptions have the same baseline rate limiting.

Solomon: Resolved with [@4f42495df6...](#)

Zenith: Verified.

[I-6] Settlement intent creation omits current global, asset-pause, and asset-eligibility gates

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/settlement/instructions/create_settlement_intent.rs#L12-L25](#)
- [../src/modules/settlement/instructions/create_settlement_intent.rs#L293-L322](#)
- [../src/modules/settlement/instructions/mint_delegated.rs#L426-L442](#)
- [../src/modules/settlement/instructions/redeem_delegated.rs#L414-L430](#)
- [spec/08_Instruction_Catalog.md#L127-L143](#)
- [spec/04_Settlement_Pathways_and_Permissions.md#L260-L265](#)

Description:

`create_settlement_intent` does not receive the global `PauseState`, the applicable `AssetPauseState`, or the collateral asset's `AssetConfig`. Although dispatch requires the settlement module to be active and the handler validates the pathway, permissions, fee policy, and optional settlement policy, it cannot evaluate the live global, asset-pause, lifecycle, extension, freshness, or asset-amount gates.

```
let pathway = PathwayPolicy::load_for_verified_pda(
    pathway_policy_account_info,
    &args.pathway_id,
    pathway_policy_bump,
)?;

pathway.assert_active()?;
pathway.assert_runtime_supported()?;
```

A permissioned creator can therefore allocate an intent while:

- the corresponding global or asset operation is paused;
- the asset is `FROZEN`;
- the asset is `WIND_DOWN` and the intent requests minting;
- the recorded extension observation is stale or incompatible with the asset or pathway policy; or

- the committed amount is outside the asset's current minimum or maximum.

A pathway pause does prevent creation because `pathway.assert_active()` is called.

The delegated and trilateral execution handlers independently evaluate the current pause and asset state before any token CPI. For example, mint execution applies:

```
asset_config.assert_extensions_fresh(clock.slot)?;  
asset_config.assert_deposits_permitted()?;  
asset_config.assert_extensions_approved()?;  
pathway.assert_collateral_extensions_allowed  
    (asset_config.observed_extension_mask)?;  
asset_config.assert_deposit_amount(asset_amount)?;
```

Consequently, an intent created under these conditions cannot bypass a live restriction or move value. It is only currently ineligible and may become eligible after a pause clears, an asset mode changes, an observation is refreshed, or configuration changes. Every execution-time permission, signature, token-account, delegation, policy, limit, and balance requirement must still pass.

No assets are escrowed during creation. The practical impact is limited to creation evidence for presently ineligible terms and voluntarily funded rent remaining in the intent until principal A cancels it or its committed expiry permits a permissionless sweep. Zero-expiry intents must be funded by principal A.

The specification describes intent creation as allocation-only, but separately states that every primary instruction should validate pause state. It does not explicitly establish whether queueing intents during temporary settlement ineligibility is intended or whether creation evidence represents current policy eligibility.

Recommendations:

It is recommended to define the intended creation semantics explicitly.

If queueing is intended, document that intent creation may occur while global, asset-pause, or `AssetConfig` gates currently prohibit execution. Clarify that `SettlementIntentCreated` proves allocation and creation-time pathway, permission, and policy validation only; it does not attest that execution is presently allowed. Client tooling should optionally preflight the current pause and asset-eligibility state before funding rent.

If creation should require current policy eligibility, add the canonical `PauseState`, `AssetPauseState`, and `AssetConfig` accounts and apply the action-specific pause, lifecycle, extension, freshness, and amount checks before allocation. Describe these checks as a creation-time snapshot rather than a guarantee of future executability, and retain every execution-time check.

Solomon: Resolved with [@8a092c00f4...](#) by documenting allocation-only intent queueing and clarifying that creation evidence does not attest current execution eligibility.

Zenith: Verified.

[I-7] Local transfer-dependent routes can be activated for non-transferable collateral

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [../src/modules/core/state/asset_config.rs#L313-L332](#)
- [../src/modules/pathway/instructions/register_pathway_policy.rs#L159-L202](#)
- [../src/modules/reserve/instructions/register_reserve_destination_with_pending.rs#L80-L183](#)
- [../src/modules/settlement/instructions/mod.rs#L353-L404](#)
- [../src/modules/settlement/instructions/mod.rs#L480-L543](#)

Description:

Chancery intentionally permits governance to approve Token-2022 collateral extensions. This can be appropriate for pathways that do not transfer local collateral. However, local pathway and reserve-destination registration do not validate whether an approved collateral mint supports the transfers required by those routes.

In particular, governance can approve an AssetConfig whose observed extension mask contains NonTransferable. A newly registered pathway initializes its forbidden_collateral_extension_mask to zero, while reserve-destination registration validates the mint, destination account, and withdrawal limits without loading the corresponding AssetConfig or checking its extensions.

The resulting configuration passes Chancery's asset and pathway extension-policy checks:

```
pub fn assert_observed_mask_allowed(
    &self,
    observed: [u64; 2],
) → Result<(), ProgramError> {
    let effective_forbidden = [
        self.forbidden_extension_mask[0]
        | collateral_default_forbidden::MINT_FORBIDDEN_LO,
        self.forbidden_extension_mask[1]
        | collateral_default_forbidden::MINT_FORBIDDEN_HI,
    ];
```

```
Self::assert_mask_subset_of_approved(
  observed,
  self.approved_extension_mask,
  effective_forbidden,
)
}
```

Local mint deposits, redeem payouts, and reserve withdrawals nevertheless move collateral through Token-2022 TransferChecked. Token-2022 rejects these transfers for non-transferable tokens. Consequently, the configured local routes cannot successfully process any collateral transfer.

Failed operations roll back atomically, so this does not cause partial deposits, burns, or withdrawals. The impact is limited to local pathway and reserve-withdrawal unavailability until governance corrects the configuration. Cross-chain pathways that do not transfer local collateral can remain compatible with the same AssetConfig.

The configuration requires explicit governance approval and activation; an unprivileged caller cannot create it. This is therefore a configuration-safety and availability issue rather than an authorization bypass.

Recommendations:

It is recommended to enforce extension compatibility according to route type. When registering or updating a local settlement pathway or reserve destination, load and verify the corresponding AssetConfig and reject NonTransferable collateral. Preserve governance approval for pathways that do not require local collateral transfers.

```
if route_requires_collateral_transfer
  && asset_config.observed_extension_mask[0] &
    extension_bit::NON_TRANSFERABLE ≠ 0
{
  return Err(ChanceryError::ForbiddenExtension.into());
}
```

Repeat the compatibility check when enabling the affected route and retain the existing execution-time extension checks as defense in depth.

Solomon: Acknowledged. None of our currently approved collateral assets—USDC, USDT, USDG, and PYUSD—has the NonTransferable extension, and it cannot be added to an already initialized mint. This issue therefore does not affect our current collateral set. But agree, when adding future assets, this should be taken into account.

Zenith: Acknowledged.

[I-8] Rebate Rounding Mode Documented Inconsistently Across Spec Sections

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- `fee_policy.rs#L515-L585: compute_internal` (rebate always uses `floored_basis_points`)
- Spec §5.5: "Percentage rebates are always rounded down... The policy `rounding_mode` applies to percentage fee assessment, not to the rebate leg."
- Spec §16.7: "Rounding is one of floor, ceiling, or nearest and applies deterministically to percentage fees and percentage rebates."

Description:

`compute_internal` always computes percentage rebates via `floored_basis_points` regardless of the policy's `rounding_mode` (line 549). `rounding_mode` is applied only to the fee assessment leg. Spec §5.5 correctly documents this. Spec §16.7 directly contradicts §5.5 and the code by stating `rounding_mode` applies to both fees and rebates.

An integration or off-chain fee simulator following §16.7 would apply the policy's `rounding_mode` to rebate calculations and produce results that diverge from on-chain behavior. For CEILING policies this means the simulated rebate would be rounded up while the actual rebate is always rounded down, causing the simulated net fee to underestimate the actual net fee charged to the user.

Recommendations:

Correct §16.7 to match §5.5 and the code: `rounding_mode` applies to percentage fee assessment only. Percentage rebates always floor.

Solomon: Resolved with [@929014d1e8...](#)

Zenith: Verified.

[I-9] One raw-input maximum is reused across collateral and issued-token denominations

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [../src/modules/core/state/asset_config.rs#L81-L88](#)
- [../src/modules/core/state/asset_config.rs#L380-L472](#)
- [../src/modules/core/instructions/register_asset.rs#L154-L184](#)
- [../src/modules/settlement/instructions/mint_direct.rs#L364-L365](#)
- [../src/modules/settlement/instructions/redeem_direct.rs#L353-L354](#)
- [spec/05_Economics_Fees_Rebates_and_Limits.md#L362-L370](#)

Description:

AssetConfig defines direction-specific minimums but only one maximum:

```
pub minimum_deposit_amount: u64,
pub minimum_redeem_amount: u64,
pub maximum_single_settlement_amount: u64,
```

Mint interprets this maximum as a collateral-input ceiling, while redeem interprets it as an issued-token-input ceiling:

```
asset_config.assert_deposit_amount(args.asset_amount)?;
asset_config.assert_redeem_amount(args.issued_token_amount)?;
```

Both guards compare their differently denominated inputs against the same raw scalar:

```
if amount > self.maximum_single_settlement_amount {
    return Err(ChanceryError::AmountExceedsMaximum.into());
}
```

Collateral registration validates an asset's stored decimals against that asset's own mint. It does not impose a universal requirement that every collateral mint have the same decimals as the issued token. Moreover, even when decimals match, non-1:1 raw-unit rates can

make the two ceilings economically different.

For a configured maximum M , mint accepts up to M collateral raw units. Redeem accepts up to M issued-token raw units and can release:

```
floor( $M \times \text{redeem\_rate\_e9} / 1,000,000,000$ )
```

collateral raw units. These amounts represent the same economic ceiling only under appropriately aligned decimals and rates.

Registration and updates verify that both direction-specific ranges contain at least one input whose conversion is nonzero and representable as $u64$. This prevents unusable arithmetic ranges but does not ensure economic equivalence between the two interpretations.

The program enforces the configured raw limit exactly, so this is not an arithmetic, authorization, or limit bypass. The ambiguity can nevertheless make one direction more permissive than governance intended while making the other unnecessarily restrictive. Additional pathway or dimension limits may mitigate the resulting exposure or impose a separate economic cap, but they do not change this field's direction-dependent meaning.

Recommendations:

It is recommended to first define the intended semantics.

If independent raw-input ceilings are required, introduce explicitly denominated fields:

```
pub maximum_mint_asset_amount: u64,  
pub maximum_redeem_issued_token_amount: u64,
```

Validate each field against its corresponding minimum and conversion range, and use it only for the matching settlement direction.

If the intention is one common collateral-value ceiling, retain one asset-denominated maximum but compare it against collateral input on mint and computed gross collateral output on redeem.

Implement any field or semantic change as an explicit pre-release ABI correction or versioned state-layout change. Update registration and mutation payloads, semantic hashes, handlers, IDL, generated clients, deployment plans, specifications, and tests.

If direction-dependent reuse is intentional, rename and document the field accordingly. Deployment tooling should display both its human-readable mint ceiling and the effective collateral-release ceiling on redeem before governance approves the configuration.

Solomon: Acknowledged. If there ever is a mismatch between token decimals, then it requires two one-way pathways with separate limits that correctly address the difference in

decimals.

Zenith: Acknowledged.

[I-10] External executor delegation violates compromised-executor containment guarantees

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/settlement/instructions/mint_delegated.rs#L699-L720](#)
- [../src/modules/settlement/instructions/redeem_delegated.rs#L581-L598](#)
- [../src/modules/settlement/instructions/mod.rs#L354-L404](#)
- [../src/modules/settlement/instructions/mod.rs#L455-L479](#)
- [tests/e2e/scenarios.ts#L428-L468](#)
- [spec/04_Settlement_Pathways_and_Permissions.md#L219-L245](#)
- [spec/02_Security_Posture_and_Authority_Control.md#L68-L75](#)
- [spec/02_Security_Posture_and_Authority_Control.md#L372-L385](#)
- [spec/05_Economics_Fees_Rebates_and_Limits.md#L344-L353](#)

Description:

The protocol's security model explicitly requires that compromise of one operational signer be survivable. For a compromised executor specifically, the documentation requires that:

- its permission can be paused;
- pathway limits cap the resulting damage; and
- it cannot route funds to arbitrary executor-owned token accounts.

The delegated-settlement implementation does not preserve these guarantees. Principal A approves the external executor as the ordinary SPL or Token-2022 delegate of each source token account. Chancery then passes the executor directly as the authority for collateral transfers and issued-token burns.

```
let received = cpi_transfer_asset_in(
  asset_token_program_account_info,
  source_asset_token_account_info,
  asset_mint_account_info,
  reserve_asset_token_account_info,
  executor_account_info,
  asset_amount,
)?;
```

```
    cpi_burn(  
        issued_token_program_account_info,  
        source_issued_token_account_info,  
        issued_token_mint_account_info,  
        executor_account_info,  
        issued_token_amount,  
    )?;
```

The CPI helpers use ordinary `invoke`; no Chancery PDA signs these source-token operations. An SPL account delegate is limited by the source account and remaining delegated amount, but it is not restricted to Chancery, a pathway, intent, instruction, destination, or time period.

A malicious or compromised executor can therefore invoke SPL Token or Token-2022 directly to transfer delegated collateral or issued tokens to any compatible token account, including an executor-owned account, or burn delegated issued tokens. Because Chancery is not invoked:

- pausing or revoking the executor's Chancery permission does not revoke its SPL allowance;
- pathway, counterparty, and executor limits do not constrain the movement;
- intent terms and destination-owner binding are not checked;
- fees and usage windows are not applied; and
- no Chancery evidence is emitted.

Ordinary delegation therefore makes the documented compromised-executor guarantees false for every source account that has approved the executor.

The implementation follows the delegated-settlement mechanism described in the pathway specification, but that design choice conflicts with the protocol's mandatory compromised-signer security posture. Similarly, the mandatory delegated counterparty daily bound limits only settlements executed through Chancery; it does not independently bound direct use of the SPL allowance.

The E2E integration approves the executor on both collateral and issued-token accounts and defaults each allowance to `u64::MAX`. This is a persistent allowance rather than a single-use authorization and is effectively unlimited relative to practical account balances until revoked, replaced, or exhausted.

The impact is limited to accounts on which the compromised executor remains the delegate and is further bounded by the account balance, remaining allowance, freeze state, and applicable token-level extensions. It does not permit unbacked minting, reserve withdrawal, or access to principals who did not delegate. Nevertheless, a compromised executor can steal or destroy the affected principal's entire approved balance, resulting in High impact.

Recommendations:

It is recommended to approve a Chancery-controlled settlement-delegate PDA instead of the external executor. The executor should remain the external signer authorizing the Chancery instruction, while Chancery signs the source-token CPI with `invoke_signed` only after every applicable pause, permission, pathway, intent, limit, destination, and fee check succeeds.

Prefer deriving the delegate PDA from the source token account to isolate each approval. Each delegated handler should:

1. verify the canonical delegate PDA and bump;
2. bind the source account to the expected owner, mint, and token program;
3. validate the executor and all settlement policy;
4. invoke the token program using the delegate PDA's signer seeds; and
5. expose no generic instruction capable of signing arbitrary token movement with that PDA.

Migration must require principals to revoke or replace every existing executor delegation. Handlers, builders, IDL, tests, and integration guidance should be updated, and users should be warned that existing executor allowances remain independently usable until revoked.

If the authority model cannot be changed immediately, exact-amount approvals, isolated source accounts funded immediately before settlement, and prompt revocation can reduce the loss ceiling. These measures do not satisfy the documented compromised-executor guarantees because direct token-program calls still bypass Chancery controls.

Solomon: Resolved with [@a41827b9da...](#) by limiting documented containment guarantees to Chancery-routed execution and documenting revocation or freezing requirements for direct SPL delegation.

Zenith: Verified.

[I-11] Per-asset rate checks do not prevent cross-asset reserve arbitrage

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [../src/modules/core/state/asset_config.rs#L292-L308](#)
- [../src/modules/core/state/asset_config.rs#L476-L529](#)
- [../src/modules/settlement/instructions/mod.rs#L594-L609](#)
- [../src/modules/settlement/instructions/mint_direct.rs#L573-L591](#)
- [../src/modules/settlement/instructions/redeem_direct.rs#L368-L370](#)
- [../src/modules/core/instructions/set_asset_mode_with_pending_change.rs#L99-L142](#)
- [scripts/deployment/lib/mainnetActivationPlan.ts#L736-L752](#)
- [scripts/deployment/lib/mainnetActivationPlan.ts#L838-L844](#)

Description:

Each AssetConfig independently requires its deposit and redeem rates to satisfy:

$$\text{deposit_rate_e9}[\text{asset}] * \text{redeem_rate_e9}[\text{asset}] \leq 10^{18}$$

This prevents a gross increase in raw asset units when settlement deposits and redeems through the same collateral. It does not protect a cycle involving two different collateral assets.

All settlement pathways use the same singleton issued-token mint. Issued tokens minted against asset A are therefore fungible with issued tokens burned to redeem asset B. The two conversions use independent configurations:

$$\begin{aligned} \text{issued_raw} &= \text{asset_a_raw} * \text{deposit_rate_e9}[A] / 10^9; \\ \text{asset_b_raw} &= \text{issued_raw} * \text{redeem_rate_e9}[B] / 10^9; \end{aligned}$$

No on-chain registry, pairwise check, common valuation epoch, or asset-specific liability partition constrains $\text{deposit_rate_e9}[A] * \text{redeem_rate_e9}[B]$.

Ignoring rounding, settlement fees, and token transfer fees, the gross cycle is economically expansive when:

```

deposit_rate[A] * redeem_rate[B] / 10^18
* 10^(decimals[A] - decimals[B])
* value(B) / value(A) > 1

```

Rounding and fees reduce the realized return but do not establish solvency. The cycle remains profitable when its gross margin exceeds those costs. Initially coherent rates can also become unsafe if one asset depegs after distinct mint and redeem pathways have been established, because settlement continues to use static governed rates without a freshness-bound common valuation.

The attack does not require two ACTIVE assets. Asset A must be ACTIVE because only that mode permits deposits. Distinct asset B may be either ACTIVE or WIND_DOWN, because both modes permit redemption. Moving B into WIND_DOWN before activating A therefore does not establish a safe collateral-rotation boundary.

The exact preconditions are:

- A has a mint pathway and permits deposits;
- B has a redeem pathway and permits redemptions;
- both pathways use the singleton issued token;
- the participant holds the required permissions;
- B's reserve has available collateral;
- the cross-pair margin exceeds applicable fees and rounding losses; and
- configured limits leave sufficient capacity.

The participant can deposit lower-value A, receive the common issued token, and burn it to withdraw greater economic value from B. In a zero-fee cycle, the participant can return issued supply to its prior level while replacing more valuable B reserves with lower-value A. With fees, the exact supply delta follows the configured fee policy, but reserve-value leakage remains possible whenever the cross-pair margin exceeds all charges.

Repetition can drain B's reserve and reduce the economic value backing issued tokens held by other users until limits, pauses, or available liquidity bind.

The initial mainnet activation plan prevents this composition by permitting a pathway only for its sole active collateral. This is an off-chain launch restriction rather than an enduring on-chain invariant. Subsequent rate writes and asset activation require governance and timelocks, reducing likelihood, but each transition evaluates only one asset. Later pathway creation, collateral rotation, or a depeg affecting an existing cross-pair can therefore establish the unsafe composition without violating the per-asset invariant.

Recommendations:

It is recommended to permit only one collateral economic domain to participate in settlement until a complete multi-asset accounting model is implemented.

Introduce a singleton on-chain collateral-selection state identifying the sole collateral mint accepted by settlement. Select it during initialization or governed activation and make it immutable under the interim model. Other AssetConfig accounts may exist for observation and future configuration, but they must not become settlement-eligible.

Every pathway registration and every mint, redeem, and collateral-consuming cross-chain handler should verify the selected mint. Execution-time checks are required so previously registered pathways cannot bypass the restriction:

```
if pathway.asset_mint != collateral_selection.selected_asset_mint {
  return Err(ChanceryError::CollateralNotSelected.into());
}
```

This restriction must cover both mint and redeem pathways. Rejecting only a second ACTIVE asset is insufficient because WIND_DOWN remains redeem-enabled.

If collateral rotation is required, implement it as a dedicated governance-controlled migration rather than an ordinary asset-mode transition. Rotation should require either:

- zero issued supply and settlement of the previous reserve; or
- proof under a defined valuation model that the replacement reserve safely backs all outstanding common liabilities.

If neither condition can be proven on-chain, use a new issued-token domain instead of changing the selected collateral.

Before enabling concurrent multi-asset settlement, implement one of the following:

- normalize every collateral to a freshness-bound common valuation and enforce constraints across every mint-enabled A and redeem-enabled B;
- establish conservative per-asset mint and redeem bounds around a canonical issued-unit value; or
- partition issued liabilities and reserves into non-fungible, asset-specific tranches.

Add on-chain coverage for selection enforcement, stale pathways, ACTIVE-to-WIND_DOWN rotation, differing decimals, and zero and nonzero fees. Add economic property tests for cross-pair rate mismatches and external value changes. If an oracle-based model is introduced, additionally test stale valuation, epoch mismatch, and depeg handling.

Solomon: Acknowledged. Chancery intentionally settles approved stablecoins at a governed 1:1 rate; it does not enforce or guarantee external market prices. Any depeg is an external collateral risk handled through limits and pause controls.

Zenith: Acknowledged.

[I-12] Range Check Allows Minimum Amount That Always Reverts

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- `asset_config.rs#L405-L448 — assert_amount_rate_ranges_executable`

Description:

When an admin registers or updates an asset, `assert_amount_rate_ranges_executable` validates that the configured rate and amount bounds produce at least one executable settlement. The check verifies that the maximum amount doesn't yield zero output (`maximum_output = 0`) and that the minimum amount doesn't overflow (`smallest_output > u64::MAX`), but it does not verify that the minimum amount itself produces a nonzero output (`smallest_output = 0` is unchecked).

Integer division floors the result: `floor(minimum_amount × rate_e9 / 109)`. When `rate_e9` is small relative to `109 / minimum_amount`, the output rounds to zero. The range check passes at registration because larger amounts within the range do produce nonzero output, but any user submitting the exact minimum amount hits a `NetOutputZero` revert at execution time.

Recommendations:

Add `smallest_output = 0` to the existing check:

```
if smallest_output == 0 || smallest_output > u64::MAX as u128 ||
    maximum_output == 0 {
    return Err(ChanceryError::AssetEconomicsInvariantViolated.into());
}
```

This forces the admin to set a `minimum_amount` high enough that the smallest allowed settlement produces at least 1 unit of output at the configured rate.

Solomon: Resolved with [PR-97](#)

Zenith: Verified.

[I-13] The fee-policy validity predicate accepts the reserved zero ID

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../src/modules/fees/state/fee_policy.rs#L308-L389](#)
- [../src/modules/pathway/state/pathway_policy.rs#L333-L337](#)
- [../src/modules/pathway/reference_validation.rs#L44-L79](#)
- [../src/modules/fees/instructions/register_fee_policy.rs#L91-L145](#)

Description:

The protocol reserves `[0; 32]` as the sentinel indicating that a pathway has no fee policy:

```
pub fn has_fee_policy(&self) → bool {
    self.fee_policy_id ≠ [0u8; 32]
}
```

Pathway reference validation therefore requires the default placeholder account when `fee_policy_id` is zero. Settlement handlers also load and apply a fee policy only when `has_fee_policy()` returns true.

However, the in-scope `FeePolicy::assert_parameter_sanity` validity predicate does not reject the reserved ID. The registration handler derives and allocates a PDA from any supplied ID, stores that ID, and then invokes the incomplete predicate:

```
let (expected_key, bump) = Pubkey::find_program_address(
    &[seeds::FEE_POLICY, args.fee_policy_id.as_ref()],
    &program_id,
);

if fee_policy_account_info.data_is_empty() {
    create_pda_account(/* ... */)?;
}

p.fee_policy_id = args.fee_policy_id;
p.assert_parameter_sanity()?;
```

Consequently, operations and a willing payer can successfully initialize the canonical zero-ID fee-policy PDA using otherwise valid parameters. Registration strips the active flag, but later configuration cannot make the account govern a pathway because its immutable identity remains the reserved absence sentinel.

Under the current instruction surface, no close or decommission path exists for the resulting account. The effect is limited to one unreachable configuration account and its payer-funded rent. It does not alter settlement calculations, enable permissionless state growth, deny service to valid pathways, or place protocol funds at risk.

Recommendations:

It is recommended to reject the reserved identity both in the registration handler before PDA derivation or allocation and defensively in the shared state validity predicate.

```
// register_fee_policy
if args.fee_policy_id == [0u8; 32] {
  return Err(ChanceryError::FeePolicyInvalidParameters.into());
}

// FeePolicy::assert_parameter_sanity
if self.fee_policy_id == [0u8; 32] {
  return Err(ChanceryError::FeePolicyInvalidParameters.into());
}
```

Add unit coverage for `assert_parameter_sanity` and registration coverage proving that zero is rejected while nonzero policy IDs remain valid.

No dedicated cleanup instruction is necessary unless the zero-ID PDA already exists on a relevant deployment. Because the original payer is not stored and only one account can be affected, leaving an existing account inert may be safer than adding a new privileged close surface.

Solomon: Resolved with [@a563a75647...](#) by rejecting the reserved zero fee-policy ID before account allocation and in the shared parameter-sanity check, with regression coverage.

Zenith: Verified.